

Databases – seminar 2

SQL*PLUS core

SQL – Structured Query Language is implemented in most Database Management Systems (DBMS). In Oracle the SQL core is implemented with some particularities.

1. Data Type:

Type	Characteristics
VARCHAR2(size)	variable length string
NUMBER(p,s)	Numeric: p (precision) = 1..38, s (scale) = -84..127
LONG	string length > 2GB
DATE	calendar date
RAW(size)	binary data. The maximum size is 2000 bytes.
LONG RAW	variable length > 2GB stores graphics, sounds, documents
ROWID	the address of each row in the table
CHAR(size)	fixed length string
BLOB	binary large object (max 4GB) stores unstructured data (text, image, video, spatial data)
BFILE	contains a pointer to a binary file stored outside the database

2. SQL operators

Operator	Characteristics
<, >, =, >=, <=, NOT	Comparison operators
BETWEEN ... AND	between two values (inclusive).
...	
IN(list)	equal to any value in the list
LIKE	similar to a template % - as many characters; _ - a character;
IS NULL	has the NULL value

3. SQL commands

Database commands are common to all versions of SQL and are divided into the following categories:

1) DDL commands (Data Definition Language)

Command	Scope
CREATE	Creates a new object (table, user, role, etc.)
ALTER	Modifies some of the properties of an object
DROP	Removes an object from the database

2) DML commands (Data Manipulation Language)

Command	Scope
SELECT	Find records in tables or views.
DELETE	Deletes records from tables

INSERT	Adds a new record to the table
UPDATE	Changes the values of some records in the tables

3) TPL commands (Transaction Processing Language)

Command	Scope
COMMIT	Completes a transaction
ROLLBACK	Cancel a transaction
SAVEPOINT	Defines a save point, where the transaction saves all operations performed so far, before continuing the next set of changes made to the database.

4) DCL commands (Data Control Language)

Command	Scope
GRANT	Gives users the necessary rights to access and manipulate objects in the database
REVOKE	Cancels certain rights for users

LDD commands

1. CREATING TABLES – CREATE TABLE COMMAND

```
CREATE TABLE table_name
(
... defining fields and related data types
... defining integrity constraints);
```

Defining constraints - can be done at field level (*in-line*) or at table level (*out-of-line*):

General syntax:

Constraint *constraint_name* **constraint_type** [(*the fields to which the constraint applies*)]

Constraint types:

1. PRIMARY KEY:

Constraint *constraint_name* PRIMARY KEY [(*fields that form the primary key*)]

2. FOREIGN KEY:

Constraint *constraint_name* FOREIGN KEY (*foreign key field*) REFERENCES *parent_table* (*primary key field*)

3. NOT NULL:

It is defined **only** at the field level where the constraint applies:

Ex: *name* VARCHAR2(20) NOT NULL

4. UNIQUE:

Constraint *constraint_name* UNIQUE [(*unique key field*)]

5. CHECK:

Constraint *constraint_name* CHECK [(*condition on a field*)]

Pay attention! When having a 1 to M link

- The tables at the 1 end of the link are called *parent tables*, and the ones at the end of many (M) are also called *child tables*.
- The parent tables are created first and then the child tables (vice versa when deleting tables).
- Add records first in the parent tables and then in the child tables (vice versa when deleting data).

Example:

```
CREATE TABLE clients
```

```
(  
  client_id NUMBER(6) CONSTRAINT pk_client PRIMARY KEY,  
  client_name VARCHAR2(20) NOT NULL,  
  client_surname VARCHAR2(20) NOT NULL,  
  telephone VARCHAR2(20),  
  credit_limit NUMBER(9,2),  
  email VARCHAR2(30) CONSTRAINT one_email UNIQUE,  
  birth_date DATE,  
  status VARCHAR2(20),  
  gender CHAR(1),  
  CONSTRAINT ck_email CHECK (email LIKE '%@%.%')  
);
```

```
-- consider that Employees table is created
```

```
CREATE TABLE orders
```

```
(  
  order_no NUMBER(12) CONSTRAINT pk_order PRIMARY KEY,  
  data DATE NOT NULL,  
  type VARCHAR2(8),  
  client_id NUMBER(6),  
  status NUMBER(2),  
  employee_id NUMBER(6),  
  --defining the foreign key constraint for the client id and making the connection with the  
  Clients table  
  CONSTRAINT fk_client FOREIGN KEY (client_id) REFERENCES clients  
  (client_id),  
  -- defining the foreign key constraint for the employee id and making the connection with  
  the Employees table  
  CONSTRAINT fk_employee FOREIGN KEY (employee_id) REFERENCES  
  employees (employee_id)  
)
```

2. CHANGING THE STRUCTURE OF THE TABLES - ALTER command

- Modify the table structure: ADD, MODIFY, DROP COLUMN, SET UNUSED
- Modify the integrity constraints: ADD, MODIFY, DROP, DISABLE CONSTRAINT
- Rename the table: RENAME

ALTER TABLE *table_name*

- **ADD (*define columns*);**
- **MODIFY (*redefine existing columns*);**
- **DROP COLUMN *column_name*;**
- **ADD CONSTRAINT *constraint_name* CONSTRAINT_TYPE;**
- **DROP CONSTRAINT *constraint_name*;**
- **DISABLE CONSTRAINT *constraint_name*;**
- **ENABLE CONSTRAINT *constraint_name*;**
- **RENAME TO *new_table_name*;**

Example: A new field is added to the Clients table:

```
ALTER TABLE clients ADD  
(income VARCHAR2(20));
```

3. DELETING THE TABLES – DROP COMMAND

DROP TABLE *table_name* CASCADE CONSTRAINTS;